ENTERSOFT

www.entersoftaudits.io

DeFa Protocol

Smart Contract Audit – Invariant Fuzz Testing

Date: 7/03/2026

Revision history and Version Control

Version	Date	Author	Description
1.0	07/03/2026	Gurkirat Singh	Fuzzing report

Engagement overview

Entersoft was engaged by DeFa Protocol to perform an advanced invariant-based fuzz testing assessment of its smart contracts.

This engagement extends beyond traditional smart contract audits by validating protocol behaviour under adversarial, non-deterministic and edge-case conditions, with a specific focus on state integrity, economic correctness, and invariant enforcement.

Table of contents

Executive Summary

The primary objective of this engagement was to evaluate whether critical protocol invariants consistently hold under adversarial and extreme execution scenarios.

The assessment utilised property-based testing (stateful fuzzing) to generate a high volume of complex and unpredictable transaction sequences, designed to intentionally stress and break predefined system constraints. This approach enables the identification of non-obvious vulnerabilities and edge-case behaviours that are typically not discoverable through manual review or deterministic testing.

Key Results:

- Total Invariants Tested: 14
- Passed Invariants: 13
- Failed Invariants: 1
- Total Executions: 100M+

The findings indicate a flawed protocol design, highlighted by the identification of a critical invariant failure that requires immediate investigation and corrective action.

Key Findings Overview

A one-page summary table:

Finding	Severity	Impact	Status
Repayment State Failure	Critical	Blocks repayments / capital recovery	Remediation required
Dead Code	Informational	Gas inefficiency	Optional

Scope

The fuzzing engagement focused on the core protocol logic within the PoolContract, specifically assessing:

- State transitions across the lending lifecycle
- Borrower repayment mechanisms
- Lender withdrawal and claim processes
- Token accounting, precision, and mathematical integrity

Primary files in scope

- `PoolContractProperties.sol`
- `Setup.sol`
- `TargetFunctions.sol`

Methodology and Tooling

This engagement employed Invariant-Based Defensive Fuzzing, a specialised testing methodology designed to validate that critical system properties remain intact regardless of how the protocol is exercised.

Rather than manually identifying failures, this approach:

- Defines strict mathematical and state-based invariants
- Actively attempts to violate these invariants through adversarial inputs
- Simulates real-world exploit scenario's

Tooling

To ensure a high level of assurance and coverage depth, testing was conducted using multiple industry-grade fuzzing engines:

- Echidna
- Medusa

Using multiple differing fuzzing engines is a best practice, as different engines employ different strategies and coverage metrics and how fast coverage is explored, significantly reducing the probability of a bug slipping through.

Protocol Overview

The DeFa Protocol facilitates decentralised invoice factoring and lending, enabling efficient capital allocation between borrowers and liquidity providers. Borrowers are able to request financing, while lenders supply USD-denominated stablecoins into designated liquidity pools. Once a pool reaches its defined soft cap, it is executed by the protocol administrator, at which point the pooled capital is deployed to the respective borrowers. Borrowers are then required to repay their obligations in accordance with predefined terms, which include multiple interest components:

- apyToRepay
- imApyToRepay

Lenders can later claim their proportional share of repaid principal and generated yield via LP tokens.. The protocol also includes mechanisms for insurance in case of borrower default and emergency stops by the admin.

Invariant Fuzzing Methodology

This engagement employed stateful, invariant-based fuzz testing to assess whether critical protocol properties remain consistently valid under arbitrary and adversarial sequences of user interactions.

A dedicated fuzzing harness was implemented to expose protocol entry points via structured handler functions. This enabled the systematic generation of randomised, high-volume interaction sequences across multiple actors and evolving protocol states, closely simulating real-world and edge-case usage scenarios.

Properties of the system were expressed as invariants that must hold true at all times. Any violation of these invariants indicates a potential logical or economic vulnerability in the protocol.

Why this matters

Traditional unit and integration testing and audits validate intended functionality; however, they do not fully account for how protocols behave under non-deterministic, adversarial, and high-frequency conditions.

Invariant fuzz testing directly addresses this gap by ensuring that core economic and accounting guarantees remain intact, even when the protocol is subjected to extreme or unanticipated scenarios.

This is critical to maintaining:

- Capital integrity across all participants
- Fair and predictable economic outcomes
- System-wide trust in protocol behaviour under stress

Invariant fuzz testing is a critical assurance layer that validates the protocol's ability to protect capital, preserve economic integrity, and withstand adversarial conditions beyond the scope of traditional smart contract audits.

Corpus and Coverage Analysis

DeFa Protocol Coverage results

During the fuzzing campaign, the following coverage levels were achieved across active testing suites targeting the PoolContract:

- **Echidna Coverage:** 87% (52,487 executions)
- **Medusa Coverage:** 87% (51,614 executions)
- **Echidna Corpus:** 53 unique execution paths
- **Medusa Corpus:** 98 unique execution paths

Corpus definition

A corpus represents a curated set of previously discovered inputs that trigger unique execution paths within the protocol.

Coverage definition

Code coverage reflects the proportion of executable lines and logical branches exercised during the fuzzing campaign. While coverage is an important indicator of testing breadth, achieving 100% coverage is not the primary objective of advanced fuzzing engagements.

In practice, smart contracts often include administrative or low-risk functions (e.g. role-based controls, configuration updates, or read-only operations) that:

- Require privileged access (e.g. specific `msg.sender` roles), or
- Do not impact the core economic or state-transition logic of the protocol

The fuzzing suite is intentionally configured to deprioritize these code paths, often resulting in less than 100% total code coverage. However, it concurrently maintains 100% effective coverage over the intricate, state-mutating financial logic, which is the actual location of potential vulnerabilities.

Invariants Breakdown

State Machine and Lifecycle

Invariant	Description	Result	Runs
invariant_statusTracking	State transitions must stringently follow the valid lifecycle: Uninitialized → Initialized → Executed → Repayment → Closed.	🟢 PASSED	100 M+
invariant_cantParticipateAfterPoolMaturity	No deposits/participations should be allowed after the pool maturity time.	🟢 PASSED	100 M+
invariant_poolCantBeReconstructedMoreThanOnce	The pool cannot be reconstructed more than once to prevent resetting borrower debt maliciously.	🟢 PASSED	100 M+

Repayments and Debt Tracking

Invariant	Description	Result	Runs
invariant_borrowerCanRepayInInstallments	After a partial repayment, the pool state must still allow further repayments.	🔴 FAILED	100 M+
invariant_repaymentIncreasesPoolBalance	The amount of USD repaid by borrowers perfectly matches the increase in the protocol's USD holdings.	🟢 PASSED	100 M+
invariant_loanAmountDecreasesByRepaidValue	When a borrower repays, the total owed decreases exactly by the amount repaid.	🟢 PASSED	100 M+
invariant_repaymentCannotExceedTotalOwed	For each invoice, the repaid amount must never exceed the original total owed.	🟢 PASSED	100 M+

Deposits, Caps and Claims

Invariant	Description	Result	Runs
invariant_poolCannotExceedHardCap	The protocol must not allow the total amount raised to exceed the defined hard cap.	🟢 PASSED	100 M+
invariant_totalClaimsCannotExceedDepositsAndRepayments	The total USD claimed by all lenders plus yield must not exceed total deposits + borrower repayments.	🟢 PASSED	100 M+
invariant_insuranceShouldNotBeOverFunded	Amount allocated to insurance should never exceed the total principal originally lent out.	🟢 PASSED	100 M+
invariant_softCapNotReached	If the pool closes without reaching the soft cap, it must retain enough USD to refund everyone.	🟢 PASSED	100 M+

Tokenomics and Pool Shares

Invariant	Description	Result	Runs
invariant_LPTokenCantBeMoreThanDepositedAmount	Total LP tokens in circulation must not exceed the total deposited principal plus accumulated yield.	🟢 PASSED	100 M+
invariant_poolShareSumToAmountRaised	The sum of all individual lenders' pool shares must accurately equal the total amountRaised.	🟢 PASSED	100 M+
invariant_claimableAmountRoundingError	The sum of all claimable amounts should be less than or equal to the actual total, and the difference (dust) should be extremely small (bounded by the number of lenders).	🟢 PASSED	100 M+

Findings and Recommendations

1. Subsequent repayments fail after initial repayment

Severity: CRITICAL

Description: The `repayLoan` function restricts execution to requiring the pool state to strictly be `PoolStatus.Executed` (Line 381). However, the internal state machine transitions the pool to `PoolStatus.Repayment` upon the very first successful borrower repayment that includes APY.

The poolContract has 5 states defined as an enum:

```
enum PoolStatus {

   Uninitialized,  // 0 - Pool not set up yet

   Initialized,    // 1 - Pool created, accepting lender deposits

   Executed,       // 2 - Pool matured, funds disbursed to borrowers

   Repayment,      // 3 - Borrowers are repaying loans

   Closed          // 4 - Pool finished

}
```

In repayLoan() function:

```
// Line 381 - Entry requirement

require(currentState == PoolStatus.Executed, "Invalid State");



// ... repayment logic ...



// Lines 446-448 - State transition

if (repaidApy > 0 && currentState == PoolStatus.Executed) {

   currentState = PoolStatus.Repayment;
```

```
}
```

1. repayLoan requires the pool to be in Executed state (state 2)
2. After the first repayment (if any APY is paid), the state changes to Repayment (state 3)
3. Now any subsequent repayLoan calls fail because the state is no longer Executed

This means only ONE borrower can ever repay before all other borrowers are blocked!

POC

Execution Scenario

1. Borrower A calls repayLoan()

2. State = Executed → passes require

3. repayLoan sets state → Repayment

4. Borrower B attempts repayLoan()

5. require(currentState == Executed) fails

```solidity
/**
 * @notice Handler for repayLoan
 * @param borrowerIndex Index to select borrower (0 or 1)
 * @param amountSeed Random seed for repayment amount
 */
function handler_repayLoan(uint8 borrowerIndex, uint256 amountSeed) public {
    uint8 idx = borrowerIndex % uint8(borrowerAddresses.length);
    address borrower = borrowerAddresses[idx];
    console2.log("This iss the borrower selected in the handler_repayLoan", borrower);
    string memory invoiceId = invoiceIds[idx];

    _handler_repayLoan(borrower, invoiceId, amountSeed);
}

function _handler_repayLoan(address borrower, string memory invoiceId, uint256 amount) internal Checks
(borrower) {
    // Fund borrower if they don't have enough
    if (USD.balanceOf(borrower) < amount) {
        USD.mint(borrower, amount);
    }

    hevm.prank(borrower);
    USD.approve(address(pool), amount);

    hevm.prank(borrower);
    try pool.repayLoan(invoiceId, amount)  {
        console2.log("Repay successful");
    } catch Error(string memory reason) {
        console2.log("ERROR: repayLoan REVERTED with reason:", reason);
        emit ErrorBytes("repayLoan", bytes(reason));
    }
}
```

```
gurkiratsingh@Gurkirats-MacBook-Air Defa latest reaudit 26 nov fuzzing % forge test --mt test_BUGPOC_handler_executePool -
vv
[⠒] Compiling...
No files changed, compilation skipped

Ran 1 test for Fuzzing/CryticToFoundry.sol:CryticToFoundry
[PASS] test_BUGPOC_handler_executePool() (gas: 1746778)
Logs:
  Invoices selected 1
  Inside the participate internal function
  Approved is successful
  This is the amount owed to the pool BEFORE execute pool function 0
  This is the amount approved 1500000000000000000000
  This is the amount approved 2500000000000000000000
  Entered execute Pool function
  Bool success of transfer to user true
  Bool success of transfer to user true
  This is IF statement inside the execute pool function
  executePool succeeded
  This is the amount owed to the pool AFTER execute pool function 4000000000000000000000
  This is the borrower Selected index 0
  Borrower Selected in the testing 0x6000000000000000000000000000000000000000
  This is the amount to be paid by the pool 1505547945205479452054 7
  This is the current state of the protocol BEFORE FIRST loan pay 2
  This iss the borrower selected in the handler_repayLoan 0x6000000000000000000000000000000000000000
  Transfer was successful or not in repayLoan true
  The amount being transferred to the pool by the borrower is 1500000001000000000000
  Repay successful
  This is the amount owed to the pool AFTER FIRST loan pay 2509863012698630136986 3
  This is the current state of the protocol AFTER FIRST loan pay 3
  This iss the borrower selected in the handler_repayLoan 0x7000000000000000000000000000000000000000
  ERROR: repayLoan REVERTED with reason: Invalid State
  This is the amount owed to the pool AFTER SECOND loan pay 2509863012698630136986 3

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.49ms (1.45ms CPU time)

Ran 1 test suite in 4.94ms (4.49ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

First Repay successful

Second Repay Unsuccessfull

Recommended Remediation:

```
// Change line 381 from:

require(currentState == PoolStatus.Executed, "Invalid State");



// To:

require(

   currentState == PoolStatus.Executed || currentState == PoolStatus.Repayment,

   "Invalid State"

);
```

2. Dead Code In fillIndividualBuckets

Severity: Informational

Description: During the coverage and corpus analysis, it was identified that the fuzzer could never reach specific deeply nested logic inside the internal fillIndividualBuckets() function. Specifically, the exact block:

```
if (payDetails.imApyToRepay > 0 && leftOverindivisualBucket > 0) {

   if (leftOverindivisualBucket >= payDetails.imApyToRepay) {

       leftOverindivisualBucket -= payDetails.imApyToRepay;

       payDetails.imApyToRepay = 0;
```

The fillIndividualBuckets function is only ever invoked in repayLoan when the _amount provided is strictly less than the totalAmount owed (a partial payment). Because it is a partial payment, the leftOverindivisualBucket will always be exhausted while covering the preceding apyToRepay and principalToRepay. Therefore, leftOverindivisualBucket can never simultaneously have enough value remaining to be >= payDetails.imApyToRepay. The scenario where a borrower's repayment exactly clears all three buckets is already fully handled by the else statement encompassing full repayments in repayLoan.

Recommended remediation

You can accept this as dead code and remove the redundant if (leftOverindivisualBucket >= payDetails.imApyToRepay) branch from fillIndividualBuckets to optimise gas consumption.

Conclusion

The invariant fuzzing campaign identified two issues within the protocol's logic, including one critical severity vulnerability that may prevent multiple borrowers from repaying their loans following the initial repayment event.

Notwithstanding these specific findings and associated failed invariants, the remaining invariants—solvency, deposit constraints, and LP token distribution—were preserved across all tested execution sequences.

The identified issues should be remediated prior to deployment. Following remediation, it is recommended that additional fuzzing campaigns be conducted to validate the updated state transition logic and any time-dependent conditions, ensuring the protocol behaves as intended under both standard and adversarial scenarios.

Overall, invariant-based fuzz testing has proven to be an effective assurance mechanism, validating the protocol's critical economic properties while identifying subtle logical inconsistencies that are unlikely to be detected through traditional testing approaches.

ENTERSOFT

www.entersoftaudits.io

Official digital asset security provider for:

HUB71

Australia | India | Singapore | USA